🎓 جزوه طلایی سیستم‌عامل

فصل ۱۰: حافظه مجازی - بخش دوم (صفحه‌بندی درخواستی و خطای صفحه)

۱. صفحه‌بندی درخواستی (Demand Paging)

به جای اینکه کل برنامه را همان اول وارد رم کنیم، از استراتژی **«تنبل» (Lazy Swapper)** استفاده می‌کنیم.

🔹 **تعریف:** صفحات تنها زمانی از دیسک به حافظه (RAM) بارگذاری می‌شوند که **برنامه واقعاً به آن‌ها نیاز داشته باشد.**

> **Pager (صفحه‌گذار):** به جای واژه Swapper (که کل پردازه را جابجا می‌کند)، در حافظه مجازی از Pager استفاده می‌کنیم که روی تک‌تک صفحات کار می‌کند.

✅ **مزایا:**

1. I/O کمتر (چون صفحات غیرضروری بارگذاری نمی‌شوند).
2. حافظه کمتر اشغال می‌شود.
3. پاسخ‌دهی سریع‌تر.
4. امکان سرویس‌دهی به کاربران بیشتر.

۲. بیت معتبر/نامعتبر (Valid-Invalid Bit)

سخت‌افزار چطور می‌فهمد یک صفحه در رم هست یا نه؟ با یک بیت در جدول صفحه (Page Table).

- **v) Valid:** صفحه در حافظه فیزیکی موجود است (Legal & Resident).
- **i) Invalid:** دو حالت دارد:
  1. صفحه کلاً نامعتبر است (جزء فضای آدرس پردازه نیست).
  2. صفحه معتبر است اما **فعلاً در دیسک (Backing Store)** است و در رم نیست.

🚨 ۳. خطای صفحه (Page Fault) - (بسیار مهم برای امتحان)

اگر پردازه سعی کند به صفحه‌ای دسترسی پیدا کند که بیت آن i) **Invalid)** است، چه می‌شود؟ یک **تله (Trap)** به سیستم‌عامل ارسال می‌شود که به آن **خطای صفحه** می‌گویند.

🔬 *۶ گام حیاتی در رسیدگی به خطای صفحه:*

بررسی جدول داخلی: سیستم‌عامل چک می‌کند آیا این آدرس کلاً غیرمجاز است یا فقط در رم نیست؟

- *اگر غیرمجاز باشد:* پردازه خاتمه می‌یابد (Abort).
- *اگر معتبر باشد ولی در رم نباشد:* باید بارگذاری شود.

1. **یافتن قاب آزاد:** سیستم‌عامل یک قاب خالی (Free Frame) در حافظه فیزیکی پیدا می‌کند.
2. **زمان‌بندی دیسک:** دستوری صادر می‌شود تا صفحه مورد نظر از دیسک خوانده شود.
3. **خواندن در قاب:** صفحه از دیسک به داخل قاب آزاد کپی می‌شود.
4. **آپدیت جدول:** جدول صفحه به‌روزرسانی می‌شود (بیت i تبدیل به **v** می‌شود).
5. **شروع مجدد دستور:** دستوری که باعث خطا شده بود، **از اول** اجرا می‌شود (Restart Instruction).

۴. صفحه‌بندی درخواستی محض (Pure Demand Paging)

در این حالت افراطی، برنامه را **بدون هیچ صفحه‌ای** در رم شروع می‌کنیم!

- اولین دستور ⬅ خطای صفحه ⬅ بارگذاری صفحه ۱.
- دستور بعدی ⬅ خطای صفحه ⬅ بارگذاری صفحه ۲.
- و ...
- **نکته:** شاید فکر کنید این کار سیستم را کند می‌کند، اما به خاطر خاصیت **«مکان‌مندی ارجاع» (Locality of Reference)**، برنامه پس از مدتی روی مجموعه‌ای از صفحات متمرکز شده و خطای صفحه کم می‌شود.

⚙ ۵. چالش‌های سخت‌افزاری (شروع مجدد دستور)

یکی از سخت‌ترین بخش‌های حافظه مجازی، توانایی **شروع مجدد (Restart)** دستور پس از خطای صفحه است.

مشکل کجاست؟ فرض کنید یک دستور ADD دارید یا دستوری مثل انتقال بلوک (Block Move) که بخشی از کار را انجام داده و وسط کار خطای صفحه می‌دهد. اگر دستور را دوباره اجرا کنیم، ممکن است داده‌هایی که در مرحله اول تغییر کرده‌اند، دوباره تغییر کنند و نتیجه خراب شود.

راهکارها:

1. **بررسی اولیه:** قبل از شروع اجرا، چک کنیم که تمام صفحات مبدأ و مقصد در رم باشند.
2. **استفاده از ثبات موقت:** تغییرات را در ثبات‌های موقت نگه داریم و فقط وقتی مطمئن شدیم خطا نداریم، آن‌ها را اعمال کنیم.

🚀 ۶. کارایی (Performance) و فرمول طلایی EAT

این بخش معمولاً در کنکور و امتحانات مسئله عددی دارد.

زمان دسترسی مؤثر (Effective Access Time - EAT):

$${EAT = (1 - p) \times ma + p \times (\text{Page Fault Time$$

- p: احتمال خطای صفحه (بین ۰ و ۱). ما می‌خواهیم p نزدیک به صفر باشد.
- ma: زمان دسترسی به حافظه (Memory Access Time).
- **Page Fault Time:** زمان رسیدگی به خطا (بسیار زیاد است چون شامل دیسک I/O می‌شود).

مثال عددی مهم: اگر دسترسی به حافظه ۲۰۰ نانوثانیه و زمان رسیدگی به خطای صفحه ۸ میلی‌ثانیه (۸,۰۰۰,۰۰۰ نانوثانیه) باشد:

$$EAT = (1-p)(200) + p(8,000,000)$$$$EAT = 200 + 7,999,800 \times p$$

⚠ **نتیجه:** حتی اگر احتمال خطا بسیار کم باشد (مثلاً یک در هزار)، سرعت سیستم به شدت افت می‌کند. برای اینکه افت سرعت زیر ۱۰٪ باشد، باید p کمتر از 0.0000025 باشد!

۷. مدیریت فضای مبادله (Swap Space)

سیستم‌عامل چطور بین فایل‌سیستم عادی و فضای Swap تعادل برقرار می‌کند؟

1. **کپی کامل (روش قدیمی):** کل برنامه اول به Swap کپی می‌شد، بعد از آنجا لود می‌شد. (شروع برنامه کند می‌شد).

2. **روش مدرن (لینوکس/ویندوز):**
   - **متن برنامه (Code):** چون فقط خواندنی است، مستقیماً از فایل اجرایی خوانده می‌شود (نیاز به Swap ندارد). اگر حافظه کم آمد، دور ریخته می‌شود و دوباره از فایل خوانده می‌شود.
   - **حافظه ناشناس (Anonymous Memory):** پشته (Stack) و هیپ (Heap) که فایلی ندارند، در Swap Space مدیریت می‌شوند.

نکته امنیتی: سیستم‌عامل قبل از دادن یک قاب آزاد به پردازه جدید، آن را **صفر می‌کند (Zero-fill-on-demand)** تا اطلاعات پردازه قبلی لو نرود.